



React Native Master Guide — Level 8 (Complete Verbatim Edition)

Native Device Hardware & Media: Permissions Architecture, Camera, Image Picker, GPS Location & Haptic Feedback

EXPO V54 • NATIVEWIND / TAILWIND CSS • ES6+

Level 8.1: Mobile OS Permissions Architecture & Lifecycle

1. Mental Model: The 2 Permission Layers 🔒

In mobile applications, permissions are strictly governed by the operating system (iOS & Android).

Layer 1: Manifest / Config Declaration

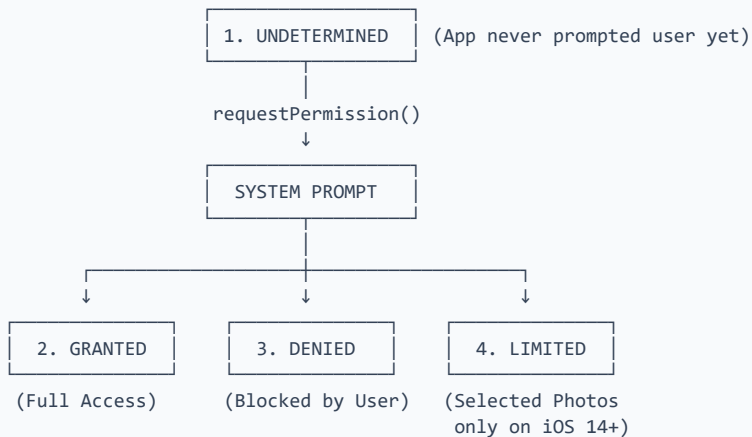
- iOS (Info.plist): `NSCameraUsageDescription`
- Android (AndroidManifest.xml): `<uses-permission />`
- 🔥 Rule: Declaring ONLY tells OS the app CAN ask for access!



Layer 2: Runtime User Grant (The OS Prompt)

- The OS displays the system dialog box when requested.
- JavaScript receives: `'undetermined'` | `'granted'` | `'denied'`

2. The 4 Permission States & The "Blocked" Trap



- **Denied vs. Blocked (Settings-Only):** Once a user permanently rejects a permission, the OS will **never show the popup again**. Calling `request...()` will immediately return `denied`.
- **The Solution (`Linking.openSettings()`):** Your app must guide the user with a direct button to open the Phone's System Settings app.
- **The Lifecycle Connection (`AppState`):** When the user switches back from Phone Settings, your app detects the `active` lifecycle event via `AppState` and **re-evaluates** the permission automatically!

Code Example: Production Permission Lifecycle & AppState

```
import React, { useState, useEffect } from 'react';
import { View, Text, Pressable, Linking, AppState } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

const App = () => {
  const [permissionStatus, setPermissionStatus] = useState('undetermined');
  const [lastCheckedTime, setLastCheckedTime] = useState('Just now');

  // 1. Silent Check (No popup)
  const checkCurrentPermission = async () => {
    setLastCheckedTime(new Date().toLocaleTimeString());
    return permissionStatus;
  };

  // 2. Contextual Permission Request
  const handleRequestPermission = async (simulatedOutcome = 'granted') => {
    if (permissionStatus === 'granted') {
      alert('Camera & Media access is ALREADY granted! ✅');
      return;
    }

    await new Promise((res) => setTimeout(res, 400));
    setPermissionStatus(simulatedOutcome);
    setLastCheckedTime(new Date().toLocaleTimeString());
  };

  // 3. Re-evaluate permission when returning from Phone Settings via AppState!
  useEffect(() => {
    const subscription = AppState.addEventListener('change', (nextAppState) => {
      if (nextAppState === 'active') {
        checkCurrentPermission();
      }
    });

    return () => subscription.remove();
  }, [permissionStatus]);

  const isGranted = permissionStatus === 'granted';
  const isDenied = permissionStatus === 'denied';
  const isLimited = permissionStatus === 'limited';

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-5 pt-3 justify-between pb-6">

        <View className="flex-1">
          <View className="mb-5">
            <Text className="text-2xl font-black text-sky-400">OS Permissions 🔒</Text>
            <Text className="text-slate-400 text-xs">
              Hardware Sandboxing, Runtime Prompts & AppState Lifecycle
            </Text>
          </View>

          { /* Permission Monitor Card */ }
          <View className="bg-slate-900 border border-slate-800 p-6 rounded-3xl mb-5 shadow-2xl">
            <Text className="text-slate-400 text-xs font-bold uppercase tracking-wider mb-2">
              Protected Hardware: Camera & Photos
            </Text>

            <View className="flex-row items-center justify-between py-2 border-b border-slate-800/80 mb-3">
              <Text className="text-white font-bold text-base">OS State:</Text>
              <View
                className={`px-3 py-1 rounded-full border ${
                  isGranted
                    ? 'bg-emerald-500/20 border-emerald-500/40'
                    : isDenied
                    ? 'bg-red-500/20 border-red-500/40'
                    : isLimited
                    ? 'bg-purple-500/20 border-purple-500/40'
                }}
              >
            </View>
          </View>
        </View>
      </SafeAreaView>
    </SafeAreaProvider>
  );
};
```

```

        : 'bg-amber-500/20 border-amber-500/40'
      }}
    >
    <Text
      className={`text-xs font-black uppercase ${
        isGranted
          ? 'text-emerald-400'
          : isDenied
            ? 'text-red-400'
            : isLimited
              ? 'text-purple-400'
              : 'text-amber-400'
      }}
    >
    {permissionStatus}
  </Text>
</View>
</View>

<View className="flex-row justify-between items-center">
  <Text className="text-slate-400 text-xs">Lifecycle Sync:</Text>
  <Text className="text-sky-400 font-mono text-xs font-semibold">
    Checked at {lastCheckedTime}
  </Text>
</View>
</View>

{/* Guided Settings Notice */}
{isDenied && (
  <View className="bg-red-500/10 border border-red-500/30 p-5 rounded-3xl mb-5 gap-3">
    <View className="flex-row items-center gap-2">
      <Text className="text-lg">⚠️</Text>
      <Text className="text-red-400 font-bold text-sm">Access Blocked by OS</Text>
    </View>
    <Text className="text-slate-300 text-xs leading-relaxed">
      The user denied permission. Calling the request dialog again is blocked by the OS. Guide the user to Phone Set
    </Text>
    <Pressable
      onPress={() => Linking.openSettings()}
      className="bg-red-500 active:bg-red-600 p-3.5 rounded-xl items-center"
    >
      <Text className="text-white font-black text-xs">
        Open Phone System Settings ⚙️
      </Text>
    </Pressable>
  </View>
)}

{isGranted && (
  <View className="bg-emerald-500/10 border border-emerald-500/30 p-5 rounded-3xl mb-5 gap-2">
    <Text className="text-emerald-400 font-bold text-sm">✅ Hardware Access Active</Text>
    <Text className="text-slate-300 text-xs">
      Your app is now authorized to launch the camera viewfinder and read native photo library assets.
    </Text>
  </View>
)}
</View>

{/* Controls */}
<View className="bg-slate-900 border border-slate-800 p-5 rounded-3xl gap-3 shadow-2xl">
  <Text className="text-slate-300 text-xs font-bold uppercase tracking-wider">
    Simulate OS Permission Outcomes:
  </Text>

  <View className="flex-row gap-2">
    <Pressable
      onPress={() => handleRequestPermission('granted')}
      className="flex-1 bg-emerald-500 active:bg-emerald-600 p-3 rounded-xl items-center"
    >
      <Text className="text-slate-950 font-black text-xs">Allow ✅</Text>
    </Pressable>

    <Pressable

```

```
        onPress={() => handleRequestPermission('denied')}
        className="flex-1 bg-red-500 active:bg-red-600 p-3 rounded-xl items-center"
      >
        <Text className="text-white font-black text-xs">Deny 🚫</Text>
      </Pressable>
    </View>
  </View>

</SafeAreaView>
</SafeAreaProvider>
);
};

export default App;
```

🧠 Telugu + English Explanation:

- **Manifest vs Runtime:** `AndroidManifest.xml` lo `<uses-permission>` pettina ventane access raadu. User app vaaduthunappudu screen meeda popup choopinchali.
- **AppState Lifecycle:** User Phone Settings loki velli permission "Allow" chesi mana app loki return raagane (`nextAppState === 'active'`), mana app automatic ga status re-check chesi UI ni update chesthundi!
- **Linking.openSettings():** Permission block ayinappudu user ki instructions ivvadam tho paatu, direct ga phone settings open chese button ivvali.

📷 **Level 8.2: Media Library, Image Picker & Camera Capture (expo-image-picker)**

1. The 2 Methods for Media Access 📱

Method	What it launches	Use Case
<code>launchImageLibraryAsync(options)</code>	Phone's Photo Gallery Roll	Uploading existing profile photos, receipt attachments, or documents.
<code>launchCameraAsync(options)</code>	Live Native Camera Shutter	Taking a fresh live selfie, scanning ID cards, or capturing live photos.

2. The Big Mobile Memory Trap: File URI vs. Base64 ⚡

❌ Option A: Base64 String (Memory Disaster)

- Converts the 48 MP photo into a 60 MB text string in JavaScript memory.
- Passing 60 MB strings over the bridge causes severe frame drops & Out-Of-Memory (OOM) app crashes!

vs.

✅ Option B: Native File URI (Zero-RAM Pointer)

- The OS writes the JPEG photo directly to the phone disk.
- JavaScript only holds a 40-character path string: `'file:///var/mobile/Containers/Data/.../photo.jpg'`
- 0 MB JavaScript RAM overhead! Ultra fast & smooth! ⚡

3. Essential Production Configuration Options

```
const result = await ImagePicker.launchImageLibraryAsync({
  mediaTypes: ['images'], // or ['images', 'videos']
  allowsEditing: true,    // ✂ Opens built-in native crop window before saving!
  aspect: [1, 1],        // ⚠ Forces 1:1 square crop (Ideal for profile avatars!)
  quality: 0.7,          // 📄 Compresses 15 MB raw camera file down to ~1 MB without visual loss!
});
```

Code Example: Complete Profile Avatar & Media Picker

```
import React, { useState } from 'react';
import { View, Text, Pressable, Image, ActivityIndicator } from 'react-native';
import { SafeAreaView, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import * as ImagePicker from 'expo-image-picker';

const MediaPlayerScreen = () => {
  const [imageUri, setImageUri] = useState(null);
  const [isProcessing, setIsProcessing] = useState(false);

  // 1. Pick from Gallery
  const handlePickFromGallery = async () => {
    const { status } = await ImagePicker.requestMediaLibraryPermissionsAsync();
    if (status !== 'granted') {
      alert('Photo Library permission is required! ⚠️');
      return;
    }

    setIsProcessing(true);
    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ['images'],
      allowsEditing: true,
      aspect: [1, 1], // 1:1 Square Crop
      quality: 0.7, // 70% Compression
    });

    if (!result.canceled) {
      setImageUri(result.assets[0].uri); // Native File URI pointer
    }
    setIsProcessing(false);
  };

  // 2. Capture with Live Camera
  const handleCaptureFromCamera = async () => {
    const { status } = await ImagePicker.requestCameraPermissionsAsync();
    if (status !== 'granted') {
      alert('Camera hardware permission is required! ⚠️');
      return;
    }

    setIsProcessing(true);
    const result = await ImagePicker.launchCameraAsync({
      allowsEditing: true,
      aspect: [1, 1],
      quality: 0.7,
    });

    if (!result.canceled) {
      setImageUri(result.assets[0].uri);
    }
    setIsProcessing(false);
  };

  return (
    <SafeAreaView>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-5 pt-3 justify-between pb-6">

        <View className="flex-1">
          <View className="mb-5">
            <Text className="text-2xl font-black text-sky-400">Media Picker 📷</Text>
            <Text className="text-slate-400 text-xs">
              expo-image-picker, Native File URIs & Image Compression
            </Text>
          </View>

          { /* Avatar Preview */ }
          <View className="bg-slate-900 border border-slate-800 p-6 rounded-3xl items-center shadow-2xl mb-5">
            <View className="relative mb-4">
              {imageUri ? (
                <Image
```

```

        source={{ uri: imageUri }}
        className="w-36 h-36 rounded-full border-4 border-sky-400/30"
      />
    ) : (
      <View className="w-36 h-36 rounded-full bg-slate-800 border-4 border-slate-700 items-center justify-center">
        <Text className="text-slate-400 text-xs font-bold">No Photo Selected</Text>
      </View>
    )
  }
</View>

<Text className="text-white font-extrabold text-lg">Profile Avatar</Text>
<Text className="text-slate-400 text-xs">
  {imageUri ? 'Native File URI Active  ' : 'Choose an option below'}
</Text>
</View>
</View>

{/* Action Controls */}
<View className="flex-row gap-2">
  <Pressable
    disabled={isProcessing}
    onPress={handleCaptureFromCamera}
    className="flex-1 bg-sky-500 active:bg-sky-600 p-4 rounded-2xl items-center justify-center shadow-lg"
  >
    {isProcessing ? (
      <ActivityIndicator size="small" color="#ffffff" />
    ) : (
      <Text className="text-white font-black text-xs">Capture Photo  </Text>
    )}
  </Pressable>

  <Pressable
    disabled={isProcessing}
    onPress={handlePickFromGallery}
    className="flex-1 bg-indigo-500 active:bg-indigo-600 p-4 rounded-2xl items-center justify-center shadow-lg"
  >
    {isProcessing ? (
      <ActivityIndicator size="small" color="#ffffff" />
    ) : (
      <Text className="text-white font-black text-xs">Pick Gallery  </Text>
    )}
  </Pressable>
</View>

</SafeAreaView>
</SafeAreaProvider>
);
};

export default MediaPickerScreen;

```

🧑 Telugu + English Explanation:

- **allowsEditing: true:** User photo theeyagane or select cheyagane OS direct ga crop window open chesthundi (square lo crop cheyochu).
- **quality: 0.7:** High-end phones lo photo theesthe 15 MB to 20 MB size untundi. **quality: 0.7** pettadam valla visual clarity taggakunda file size **1 MB ki compress** aypothundi.
- **File URI:** JavaScript memory lo photo data pettakunda, phone hard disk lo save ayina file address (**file:///...**) theskuntam. Deeni valla app 100% smooth ga run avthundi.

📍 Level 8.3: Location, GPS Tracking & Reverse Geocoding (expo-location)

1. Foreground vs. Background Location Permissions 📱

1. Foreground Location (While App Is In Use)

- `Location.requestForegroundPermissionsAsync()`
- Allowed ONLY while the app is actively on the screen.
- Use case: Showing nearby restaurants on Zomato/Swiggy.

vs.

2. Background Location (Always Allowed / In Background)

- `Location.requestBackgroundPermissionsAsync()`
- Tracks GPS even when the phone is locked in your pocket!
- Strict Store Guidelines (Google/Apple heavily review!).
- Use case: Uber driver navigation, fitness run tracking.

2. Reverse Geocoding & Distance Filtering 📍

- **Reverse Geocoding:** Converts raw coordinates `{ latitude: 17.4483, longitude: 78.3915 }` into human-readable street addresses: `"HITEC City, Hyderabad, 500081"`.
- **Battery Optimization (`distanceInterval`):** Prevents constant GPS waking by triggering updates only after moving 10+ meters:

```
const subscription = await Location.watchPositionAsync(
  {
    accuracy: Location.Accuracy.Balanced, // Battery balance
    timeInterval: 2000,
    distanceInterval: 10, // ⚡ Only notify if moved 10+ meters!
  },
  (location) => {
    console.log('Coordinates:', location.coords);
  }
);
```


Code Example: GPS Tracking & Reverse Geocoding

```
import React, { useState, useEffect } from 'react';
import { View, Text, Pressable, ActivityIndicator } from 'react-native';
import { SafeAreaView, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import * as Location from 'expo-location';

const LocationScreen = () => {
  const [coords, setCoords] = useState(null);
  const [address, setAddress] = useState(null);
  const [isLiveTracking, setIsLiveTracking] = useState(false);
  const [isFetching, setIsFetching] = useState(false);

  // 1. Single Snapshot
  const fetchSingleLocation = async () => {
    const { status } = await Location.requestForegroundPermissionsAsync();
    if (status !== 'granted') {
      alert('Location permission is required! 🚨');
      return;
    }

    setIsFetching(true);
    const location = await Location.getCurrentPositionAsync({
      accuracy: Location.Accuracy.High,
    });
    setCoords(location.coords);

    const reverse = await Location.reverseGeocodeAsync({
      latitude: location.coords.latitude,
      longitude: location.coords.longitude,
    });
    if (reverse.length > 0) {
      setAddress(`${reverse[0].street}, ${reverse[0].city}, ${reverse[0].postalCode}`);
    }
    setIsFetching(false);
  };

  // 2. Continuous Watcher
  useEffect(() => {
    let subscription = null;

    const startWatching = async () => {
      const { status } = await Location.requestForegroundPermissionsAsync();
      if (status !== 'granted') return;

      subscription = await Location.watchPositionAsync(
        {
          accuracy: Location.Accuracy.Balanced,
          TimeInterval: 2000,
          distanceInterval: 10,
        },
        async (loc) => {
          setCoords(loc.coords);
          const reverse = await Location.reverseGeocodeAsync({
            latitude: loc.coords.latitude,
            longitude: loc.coords.longitude,
          });
          if (reverse.length > 0) {
            setAddress(`${reverse[0].street}, ${reverse[0].city}, ${reverse[0].postalCode}`);
          }
        }
      );
    };

    if (isLiveTracking) startWatching();

    return () => {
      if (subscription) subscription.remove();
    };
  }, [isLiveTracking]);

  return (
```

```

<SafeAreaView>
  <StatusBar style="light" backgroundColor="#090d16" />
  <SafeAreaView className="flex-1 bg-slate-950 px-5 pt-3 justify-between pb-6">

    <View className="flex-1">
      <View className="mb-5">
        <Text className="text-2xl font-black text-sky-400">GPS & Location 📍 </Text>
        <Text className="text-slate-400 text-xs">
          expo-location, Live Subscriptions & Reverse Geocoding
        </Text>
      </View>

      <View className="bg-slate-900 border border-slate-800 p-5 rounded-3xl mb-4 shadow-2xl">
        <Text className="text-slate-400 text-xs font-bold uppercase mb-2">Live GPS Coordinates</Text>
        {coords ? (
          <View className="bg-slate-950 p-4 rounded-2xl border border-slate-800 mb-3">
            <Text className="text-sky-400 font-mono text-sm font-bold">
              Lat: {coords.latitude.toFixed(4)}° N
            </Text>
            <Text className="text-sky-400 font-mono text-sm font-bold mt-1">
              Long: {coords.longitude.toFixed(4)}° E
            </Text>
          </View>
        ) : (
          <View className="bg-slate-950 p-4 rounded-2xl border border-slate-800 mb-3 items-center">
            <Text className="text-slate-500 text-xs">No GPS data loaded yet</Text>
          </View>
        )}

        {address && (
          <View className="bg-sky-500/10 border border-sky-500/30 p-3.5 rounded-2xl">
            <Text className="text-sky-400 font-bold text-xs uppercase mb-1">📍 Street Address:</Text>
            <Text className="text-white font-extrabold text-sm">{address}</Text>
          </View>
        )}
      </View>
    </View>

    <View className="gap-2.5">
      <Pressable
        onPress={() => setIsLiveTracking((prev) => !prev)}
        className={`p-4 rounded-2xl items-center justify-center shadow-lg ${
          isLiveTracking ? 'bg-amber-500 active:bg-amber-600' : 'bg-emerald-500 active:bg-emerald-600'
        }`}
      >
        <Text className="text-slate-950 font-black text-xs uppercase">
          {isLiveTracking ? 'Stop Live GPS Tracking 🛑' : 'Start Live GPS Watcher (watchPositionAsync) 📍'}
        </Text>
      </Pressable>

      <Pressable
        disabled={isFetching || isLiveTracking}
        onPress={fetchSingleLocation}
        className="bg-slate-900 border border-slate-800 p-3.5 rounded-xl items-center"
      >
        {isFetching ? (
          <ActivityIndicator size="small" color="#38bdf8" />
        ) : (
          <Text className="text-sky-400 font-bold text-xs">
            Fetch Single Position (getCurrentPositionAsync) 📍
          </Text>
        )}
      </Pressable>
    </View>

  </SafeAreaView>
</SafeAreaView>
);
};

export default LocationScreen;

```

Telugu + English Explanation:

- **getCurrentPositionAsync vs watchPositionAsync:** `getCurrent...` ఒక్కసారి GPS check chesi chip ni off chestundi. `watch...` live map movement kosam continuous ga GPS coordinates ni stream chestundi.
- **distanceInterval: 10:** User కనీసం 10 మీటర్లు move ayitheనే GPS calculate cheyali ani rule pedatham, deeni valla phone battery drain avvadu.

 Level 8.4: Haptic Feedback & Tactile UX (expo-haptics)

1. The 3 Haptic Engine Primitives

Method	What it feels like	Best Use Case
<code>Haptics.impactAsync(style)</code>	Sharp mechanical tap / click.	Button presses, toggle switches, tab navigation.
<code>Haptics.notificationAsync(type)</code>	Distinct semantic vibration rhythm.	Payment success, wrong password shake, warning alerts.
<code>Haptics.selectionAsync()</code>	Ultra-light micro-tick.	Scrolling through date pickers, number steppers, sliders.

Code Example: Tactile Soundboard & Stepper Micro-Ticks

```
import React, { useState } from 'react';
import { View, Text, Pressable } from 'react-native';
import { SafeAreaView, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import * as Haptics from 'expo-haptics';

const HapticsScreen = () => {
  const [stepper, setStepper] = useState(5);

  // 1. Button Mechanical Taps
  const handleButtonTap = (style) => {
    Haptics.impactAsync(style);
  };

  // 2. Payment & Error Alerts
  const handlePaymentSuccess = () => {
    Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
    alert('Payment Successful! ₹500 transferred 🏧');
  };

  const handleWrongPassword = () => {
    Haptics.notificationAsync(Haptics.NotificationFeedbackType.Error);
    alert('Invalid Passcode! Please try again ❌');
  };

  // 3. Stepper Wheel Micro-Tick
  const handleStep = (delta) => {
    Haptics.selectionAsync(); // Ultra-light tactile tick
    setStepper((prev) => Math.max(0, Math.min(10, prev + delta)));
  };

  return (
    <SafeAreaView>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-5 pt-3 justify-between pb-6">

        <View className="flex-1">
          <View className="mb-5">
            <Text className="text-2xl font-black text-sky-400">Haptics & Tactile UX 🏠 </Text>
            <Text className="text-slate-400 text-xs">
              expo-haptics: Mechanical Impacts, Notifications & Wheel Ticks
            </Text>
          </View>

          {/* Section 1: Impacts */}
          <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl mb-4 gap-2.5">
            <Text className="text-white font-bold text-xs uppercase">1. Mechanical Impacts:</Text>
            <View className="flex-row gap-2">
              <Pressable
                onPress={() => handleButtonTap(Haptics.ImpactFeedbackStyle.Light)}
                className="flex-1 bg-slate-800 p-3 rounded-xl items-center">
                <Text className="text-sky-400 font-bold text-xs">Light 🦟 </Text>
              </Pressable>
              <Pressable
                onPress={() => handleButtonTap(Haptics.ImpactFeedbackStyle.Medium)}
                className="flex-1 bg-slate-800 p-3 rounded-xl items-center">
                <Text className="text-indigo-400 font-bold text-xs">Medium 🕳 </Text>
              </Pressable>
              <Pressable
                onPress={() => handleButtonTap(Haptics.ImpactFeedbackStyle.Heavy)}
                className="flex-1 bg-slate-800 p-3 rounded-xl items-center">
                <Text className="text-purple-400 font-bold text-xs">Heavy 🦶 </Text>
              </Pressable>
            </View>
          </View>

          {/* Section 2: Notifications */}
          <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl mb-4 gap-2.5">
```

```

<Text className="text-white font-bold text-xs uppercase">2. App Notifications:</Text>
<View className="flex-row gap-2">
  <Pressable
    onPress={handlePaymentSuccess}
    className="flex-1 bg-emerald-500/20 border border-emerald-500/40 p-3 rounded-xl items-center"
  >
    <Text className="text-emerald-400 font-bold text-xs">Success ✅</Text>
  </Pressable>
  <Pressable
    onPress={handleWrongPassword}
    className="flex-1 bg-red-500/20 border border-red-500/40 p-3 rounded-xl items-center"
  >
    <Text className="text-red-400 font-bold text-xs">Error ❌</Text>
  </Pressable>
</View>
</View>

{/* Section 3: Stepper Tick */}
<View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl gap-2">
  <Text className="text-white font-bold text-xs uppercase">3. Selection Stepper (Micro-Ticks):</Text>
  <View className="bg-slate-950 p-3 rounded-xl border border-slate-800 flex-row justify-between items-center">
    <Pressable
      onPress={() => handleStep(-1)}
      className="w-10 h-10 bg-slate-800 rounded-xl items-center justify-center"
    >
      <Text className="text-white font-bold text-base">-</Text>
    </Pressable>
    <Text className="text-white font-black text-xl">{stepper}</Text>
    <Pressable
      onPress={() => handleStep(1)}
      className="w-10 h-10 bg-slate-800 rounded-xl items-center justify-center"
    >
      <Text className="text-white font-bold text-base">+</Text>
    </Pressable>
  </View>
</View>

</SafeAreaView>
</SafeAreaProvider>
);
};

export default HapticsScreen;

```

🧑 Telugu + English Explanation:

- **Haptics.impactAsync**: Button tap chesinappudu physical phone lo real switch nokkinattu mechanical feel isthundi.
- **Haptics.notificationAsync(Success)**: Google Pay or PhonePe lo payment successful ayinappudu phone double-buzz ayye feel ni create chestundi.
- **Haptics.selectionAsync**: Date picker or quantity stepper scroll chestunappudu prathi number change ki subtle micro-tick vibration isthundi.